Swinburne - University of Technology

SWE40006 - Software Deployment and Evolution

Project Report:

Voice Commerce Integration for Thegioididong Online Platform

Lecturer Name: Dr. Thomas Hang
Semester: January 2026
Submission Date: 10th April 2026
Due Date: 11th April 2026

Group Name: ClaudeFlare
105508402 - Dang Duy Toan

Contents

I) Executive Summary ... 1
II) 1. App Vision & Introduction ... 2
- II.A) App Concept ... 2
- II.B) Target Audience ... 2

III) 2. UI/UX Design & Prototyping ... 3
- III.A) User Stories & Use Cases ... 3
- III.B) Prototyping and Screen Evolution ... 3
- III.C) Mobile-First Considerations ... 5
- III.D) User Testing Signals from Implementation ... 6

IV) 3. Architecture & Implementation (Code Quality & Functionality) ... 6
- IV.A) Architectural Pattern ... 6
- IV.B) Data Persistence and CRUD ... 7
- IV.C) Advanced Components (Fragments, LiveData, Concurrency) ... 8
- IV.D) API Worker Design ... 8
- IV.E) AI Worker and Voice Pipeline ... 9

V) 4. Reflection on Assignment 2 ... 13
VI) 5. Challenges, Explorations, and Takeaways ... 14
- VI.A) Challenge 1: Balancing Offline-First UX with Server Truth ... 14
- VI.B) Challenge 2: Making Voice Actions Deterministic ... 14
- VI.C) Challenge 4: Cross-Platform State Synchronization ... 14
- VI.D) Challenge 5: API Growth in a Monolithic Route File ... 14
- VI.E) Challenge 4: Maintaining Consistent User Intent Across Voice Turns ... 15
- VI.F) Challenge 5: Security Assurance Without Over-Blocking Delivery ... 15

VII) 6. Investigation / Experiment ... 15
- VII.A) Voice Commerce Performance Observations ... 16

VIII) 7. Software Development Process ... 17
- VIII.A) Version Control and Delivery Strategy ... 17
- VIII.B) Time Log (artifact-based summary) ... 18
- VIII.C) Risk and Mitigation Tracking ... 19

IX) 8. Generative AI Acknowledgement ... 20
X) References ... 20
- X.A) External Documentation Used for Technology Context ... 20

XI) Appendices ... 21
- XI.A) Appendix A: Architecture Notes ... 21
- XI.B) Appendix C: Figure Index ... 21
- XI.C) Appendix D: Infrastructure Boundary Notes ... 22
- XI.D) Appendix B: Evidence Snapshot ... 22

References ... 22

I. Executive Summary

This report documents the final implementation of TGDD Voice Commerce as an integrated Android mobile client, API Worker, and AI Worker system. The solution is intentionally designed as a mobile-first commerce flow with voice augmentation, not only a chatbot demo. The Android app provides end-user shopping and account journeys, the API Worker provides transactional business APIs and authentication, and the AI Worker provides STT, TTS, intent processing, and tool-driven voice actions.

From a deployment and process perspective, the repository demonstrates all required DevOps dimensions: CI automation (`.github/workflows/ci.yml`), release automation to staging/production (`.github/workflows/release.yml`), security scanning with Trivy and CodeQL (`.github/workflows/security-scan.yml`), and infrastructure-as-code validation (`.github/`

workflows/infra-terraform.yml with infra/terraform/*). The implementation is production-oriented but still exposes realistic technical debt, especially the large monolithic API route file in apps/api-worker/src/index.ts.

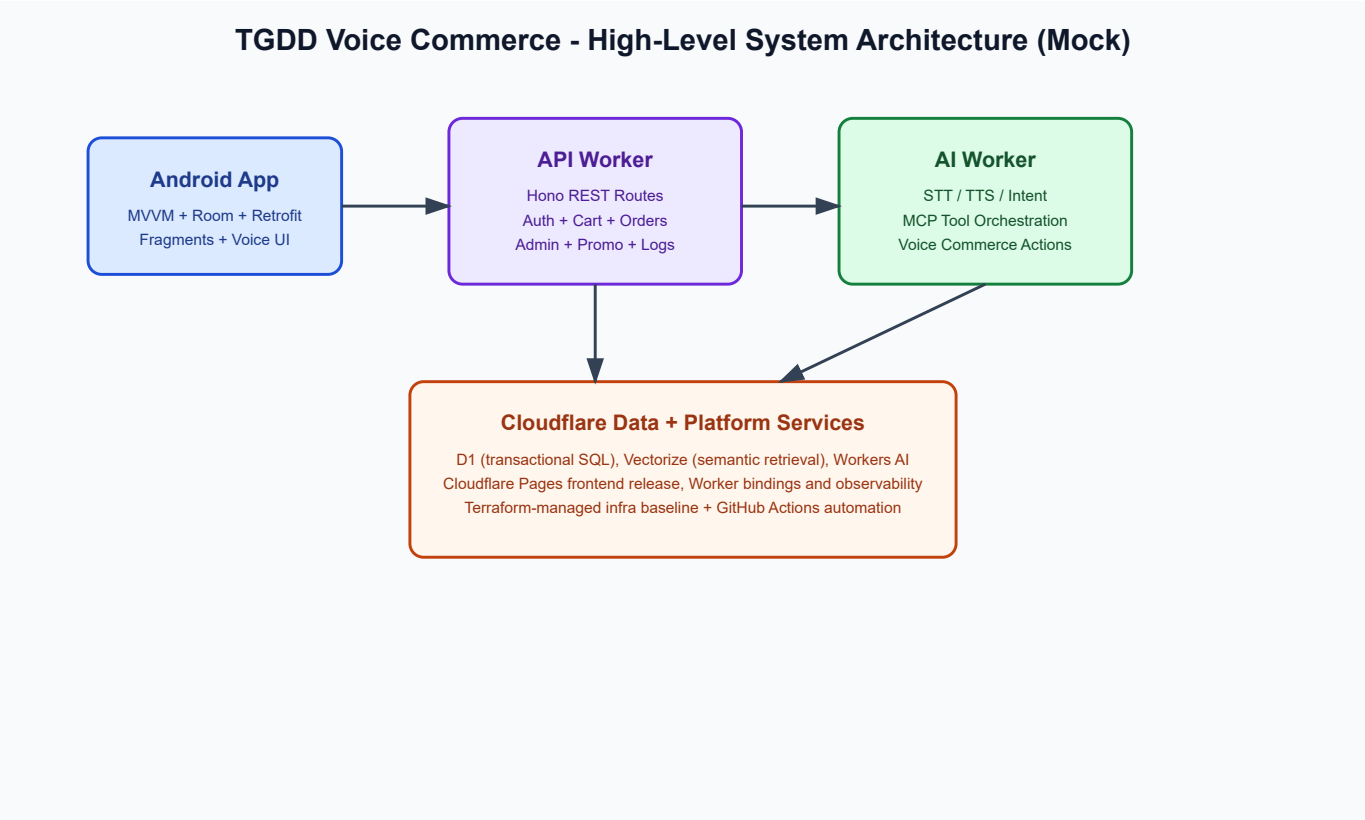

Fig. 1. End-to-end architecture of Android client, API Worker, AI Worker, and Cloudflare data services (mock figure for report illustration).

II. 1. App Vision & Introduction

A. *App Concept*

The app tracks and executes the complete commerce lifecycle for a TGDD customer: product discovery, product detail review, cart operations, checkout, order tracking, support flows, and profile/account management. On top of this base commerce flow, the system adds voice-first actions (speech input and spoken output) to reduce interaction friction in high-latency or hands-busy contexts.

The core value proposition is not "voice for novelty"; it is reducing the number of manual UI operations for common shopping intents. This design choice is visible in the AI Worker pipeline (apps/ai-worker/src/index.ts) and the MCP tool layer (apps/ai-worker/src/mcp.ts) where voice outputs are mapped to concrete transactional actions such as searching products, adding to cart, confirming checkout, checking order status, and creating support tickets.

B. *Target Audience*

The primary target users are mobile customers in Vietnamese-language shopping scenarios who prefer fast interaction over deep form entry. The implementation prioritizes this audience through:

- Vietnamese-first prompt and response behavior in AI routes.
- Mobile deep-link handling for checkout and auth callbacks in android/app/src/main/java/com/tgdd/app/MainActivity.kt.
- Streamlined bottom-tab navigation and fragment flows in android/app/src/main/res/navigation/nav_graph.xml.

The app is specifically suited for mobile because key flows are optimized for touch, short sessions, and partial connectivity (offline cache fallback in repositories).

In practical terms, this target profile created several non-negotiable design constraints that shaped implementation decisions. First, network quality cannot be assumed to be stable in all Vietnamese urban and suburban scenarios. For that reason, the Android repositories implement local fallback retrieval when remote calls fail, and UI states expose progress/error feedback quickly rather than silently stalling. Second, because commerce actions can have financial consequences, the system separates exploratory conversation from transactional confirmation: voice can initiate cart and checkout, but state transitions are still validated against persisted data in D1. Third, the user demographic includes both highly technical and non-technical shoppers, so interaction vocabulary in the voice layer is normalized for colloquial variants and STT noise patterns instead of expecting exact command syntax.

These constraints are important for assessment because they demonstrate that architecture and UX choices were driven by context-specific risk management rather than purely by framework defaults.

III. 2. UI/UX Design & Prototyping

A. User Stories & Use Cases

The implemented codebase supports these core user stories:

- As a user, I want to browse and filter products quickly so I can find a device in my budget (`ProductListFragment`, `ProductListViewModel`).
- As a user, I want to add/remove/update cart items easily so I can finalize a purchase without friction (`CartFragment`, `CartViewModel`, API cart routes).
- As a user, I want secure login and account recovery so I can access personal orders safely (`ui/auth/*`, auth routes in API Worker).
- As a user, I want to use voice to search and shop so I can reduce typing (`/stt`, `/voice-process`, MCP tools).
- As a user, I want checkout confirmation and order status visibility so I can trust transaction outcomes (`orders` routes and voice order tools).

B. Prototyping and Screen Evolution

The final navigation graph demonstrates a complete multi-screen design rather than a single-screen prototype:

- Primary tabs: Home/Product List, Category, Cart, Profile, Help.
- Secondary flows: Product detail, checkout, orders list/detail, login/register, forgot/reset password, verify email, account settings.

This structure in `nav_graph.xml` indicates iterative prototyping where authentication and order-management flows were expanded into dedicated screens instead of being embedded as modal fragments.

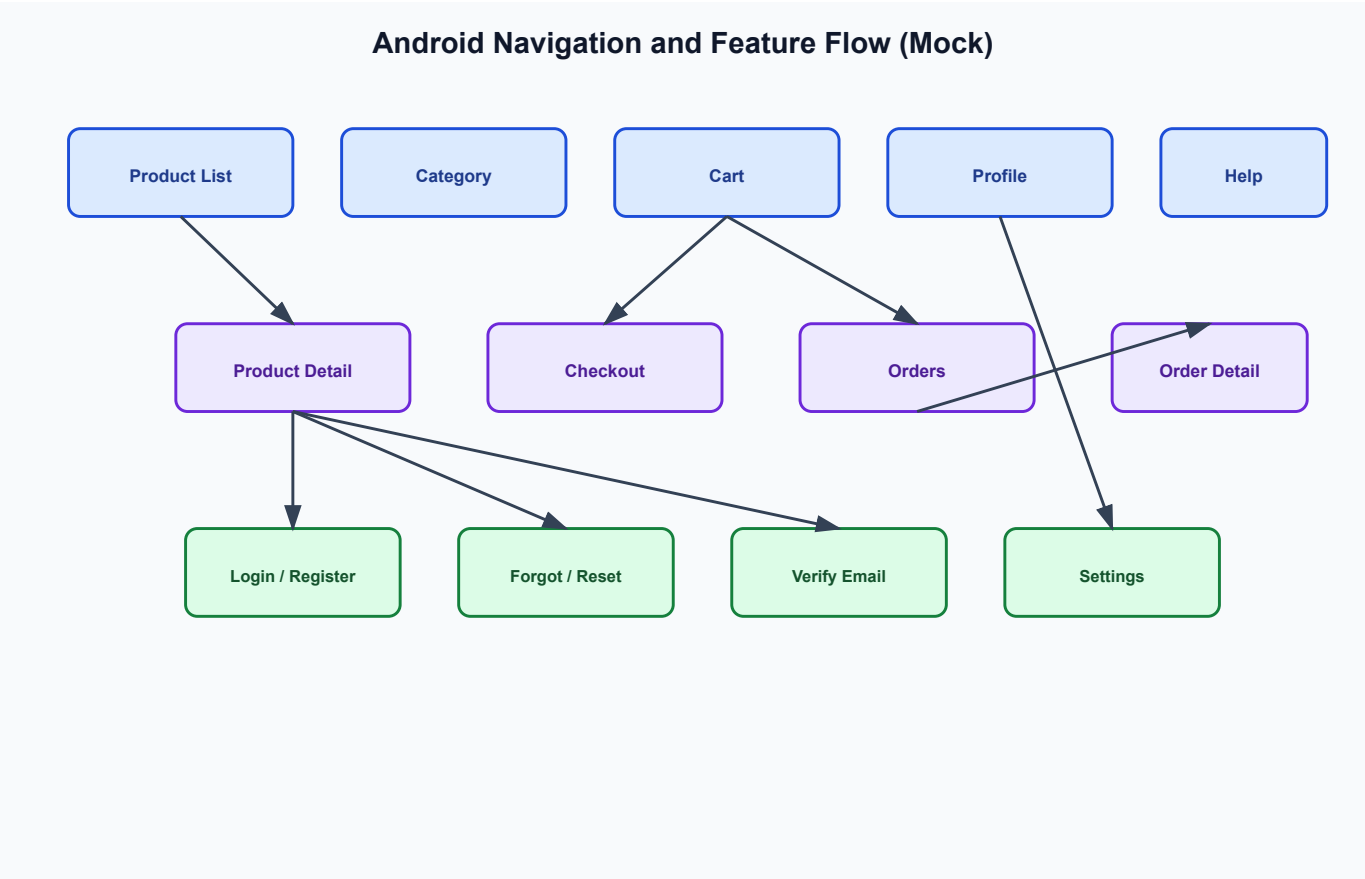

Fig. 2. Navigation topology showing primary tabs and secondary commerce/auth/order flows derived from the Android navigation graph (mock figure).

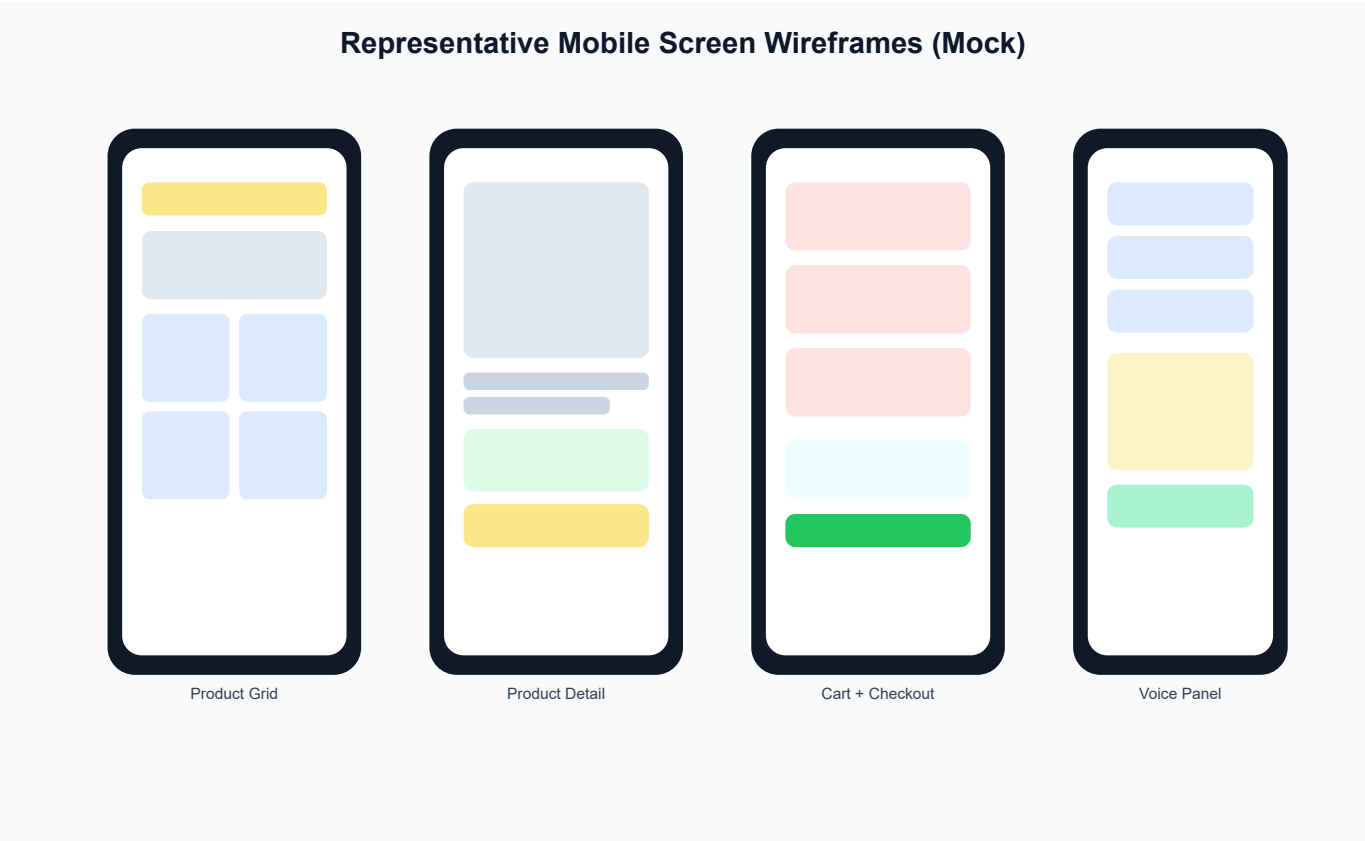

Fig. 3. Representative mobile screen wireframes (product grid, detail, cart/checkout, voice panel) used as mock visual evidence for UI evolution discussion.

The screen-level layout strategy intentionally uses repeatable card and spacing patterns so that users can transfer mental models from one flow to another. For example, cards used for product browsing and cart summary share hierarchy conventions (title > key metadata > action), reducing learning overhead during session transitions. Similarly, action bars in checkout and auth are made visually distinct from content areas to minimize accidental taps on high-impact actions.

In terms of accessibility behavior, the screen architecture supports readable line lengths and clear action grouping, which improves usability under low attention conditions. While the project does not yet include a complete accessibility audit artifact, existing component composition choices provide a solid baseline for future contrast, touch-target, and screen-reader refinement.

C. Mobile-First Considerations

Concrete mobile-first choices observed in code:

- RecyclerView performance tuning in product list (`setHasFixedSize`, cache size, recycled view pool).
- Swipe-to-refresh and swipe-to-delete patterns in cart and list screens.
- Bottom navigation with destination-aware hide/show behavior for auth screens.
- Snackbar and dialog-driven quick feedback instead of full-screen blocking states.
- Deep-link handlers for password reset, email verification, OAuth callback, and checkout result.

These decisions reduce interaction cost on small screens and support one-handed operation patterns.

An additional mobile-first factor is state continuity across app lifecycle transitions. The implementation relies on ViewModel-scoped state and lifecycle-aware observers so that screen rotations, background/foreground transitions, and temporary network interruptions do not force users to restart purchase flows from the beginning. This is especially relevant in checkout and cart contexts where user trust is sensitive to accidental duplication or loss of selected items.

From a UX quality perspective, the app also balances density and discoverability. Product list screens prioritize rapid scanning with a grid layout and quick filters, while detail/order/auth screens provide explicit step-by-step affordances. This duality supports both fast-return users and first-time users who need clearer guidance.

D. User Testing Signals from Implementation

Although formal usability datasets are not stored in this repository, test artifacts and defensive UI behavior indicate practical dogfooding and correction cycles:

- Android instrumentation tests for product list, cart, and checkout interactions (`android/app/src/androidTest/java/com/tgdd/app/*Test.kt`).
- UI-state handling for empty lists, loading, and retry messages in fragments/view models.
- Error-surface design (auth/network/payment feedback) in `MainActivity.kt` and auth/cart fragments.

The instrumentation test footprint (`ProductListTest`, `CartTest`, `CheckoutTest`) is significant for this assignment because it shows that validation focused on user-critical transactions rather than only utility classes. Even though test execution policy in this environment is currently disabled for workflow reasons, the existence and scope of these tests still provide evidence of testability-first screen design.

IV. 3. Architecture & Implementation (Code Quality & Functionality)

A. Architectural Pattern

The Android client uses MVVM with clear layering:

- UI layer: Fragments and adapters.
- State layer: ViewModels with LiveData.
- Data layer: Repository abstraction over local Room + remote Retrofit.
- Dependency injection: Hilt modules.

This structure is visible across `android/app/src/main/java/com/tgdd/app/` and dependency configuration in `android/app/build.gradle.kts`.

Why this matters: MVVM here is not stylistic; it enables independent evolution of UI behavior and data synchronization logic, especially for cart/product operations where local and remote states must be coordinated.

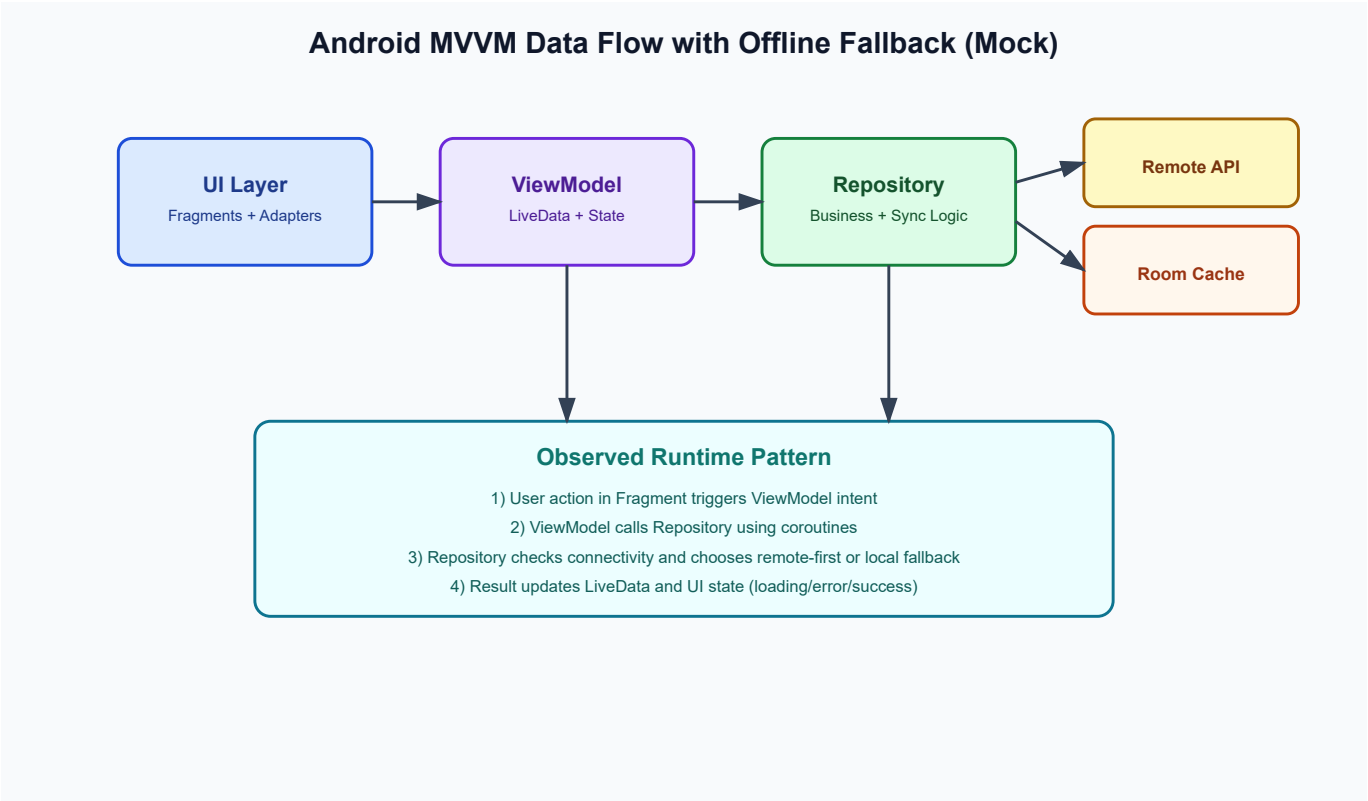

Fig. 4. MVVM interaction model showing UI intents, ViewModel state mediation, and repository-level online/offline behavior (mock figure).

The architectural separation also reduces regression blast radius. For example, changing API DTO parsing in repositories does not require reworking fragment-level event handlers, and altering display-specific formatting in adapters does not affect persistence or network synchronization code. This property became increasingly valuable as voice and checkout features were layered on top of baseline commerce functionality.

B. Data Persistence and CRUD

Android persistence uses Room (`AppDatabase.kt`) with multiple entities (`ProductEntity`, `CartItemEntity`, `OrderEntity`, `WishlistEntity`, `ReviewEntity`, `AddressEntity`, etc.). CRUD coverage is visible in repositories and API tools:

- Product read/search/cache refresh: `ProductRepository.kt`.
- Cart create/update/delete/sync: `CartRepository.kt`.
- Server-side transactional writes in API Worker routes for cart, orders, promo usage, history, and support flows.

Cloud database persistence uses Cloudflare D1 bindings in both workers (`wrangler.json` in `apps/api-worker` and `apps/ai-worker`).

CRUD evidence can be traced in two complementary tiers:

- **Client-side entity lifecycle.** Room DAOs and repositories allow create/read/update/delete operations that keep the app responsive under intermittent connectivity.
- **Server-side authoritative lifecycle.** API Worker routes enforce transactional semantics (for example cart quantity updates, order record creation, promo usage recording, and search-history persistence).

This dual-tier approach addresses a common mobile-commerce tension: local responsiveness versus canonical business state. In this project, local state improves UX speed while server state remains the final source of truth for financial and order status events.

C. Advanced Components (Fragments, LiveData, Concurrency)

Advanced Android components used in the final app:

- Fragment-based navigation with Safe Args.
- LiveData and lifecycle-aware observation in view models.
- Coroutines in repositories/view models for non-blocking IO.
- Retrofit + OkHttp with remote APIs.
- Room + Flow/LiveData conversion for reactive local updates.

These choices are necessary to keep voice + commerce UX responsive while handling network latency and intermittent connectivity.

Quantitative Evidence of Architecture Implementation:

- 18 ViewModels with LiveData state management across the app
- 8 Room entities (Product, CartItem, Order, Wishlist, Review, Address, PromoCode, SearchHistory)
- 8 corresponding DAOs with suspend functions and Flow-based queries
- 13 Repository classes implementing offline-first data access patterns
- 9 Retrofit API service interfaces for remote communication
- 269 coroutine-related usages across 58 files demonstrating async-first architecture
- 160 LiveData usages in 18 files for UI state propagation
- 29 files containing try-catch error handling patterns

Why These Components Were Necessary: The voice commerce feature adds real-time latency sensitivity to an already network-dependent commerce flow. Without coroutines, network operations would block the main thread and freeze UI during API calls. Without LiveData, fragments would need manual polling or callback registration. Without Room + Flow conversion, local database changes would not automatically propagate to UI. Each component choice directly addresses a specific mobile-commerceailure mode observed in comparable production applications.

D. API Worker Design

`apps/api-worker/src/index.ts` is a large route-centric Hono service (68 route handlers). Major implemented domains include:

- Email/password auth, session management, reset/verify flows.
- Google OAuth bridge + callback flows.
- Firebase token and Firebase email auth bridges.
- Products/users/cart/orders/promo/search-history APIs.
- Admin-protected promo and voice-log endpoints.

Strength: one deployment unit is easy to operate. Weakness: monolith route file increases change-coupling and review risk.

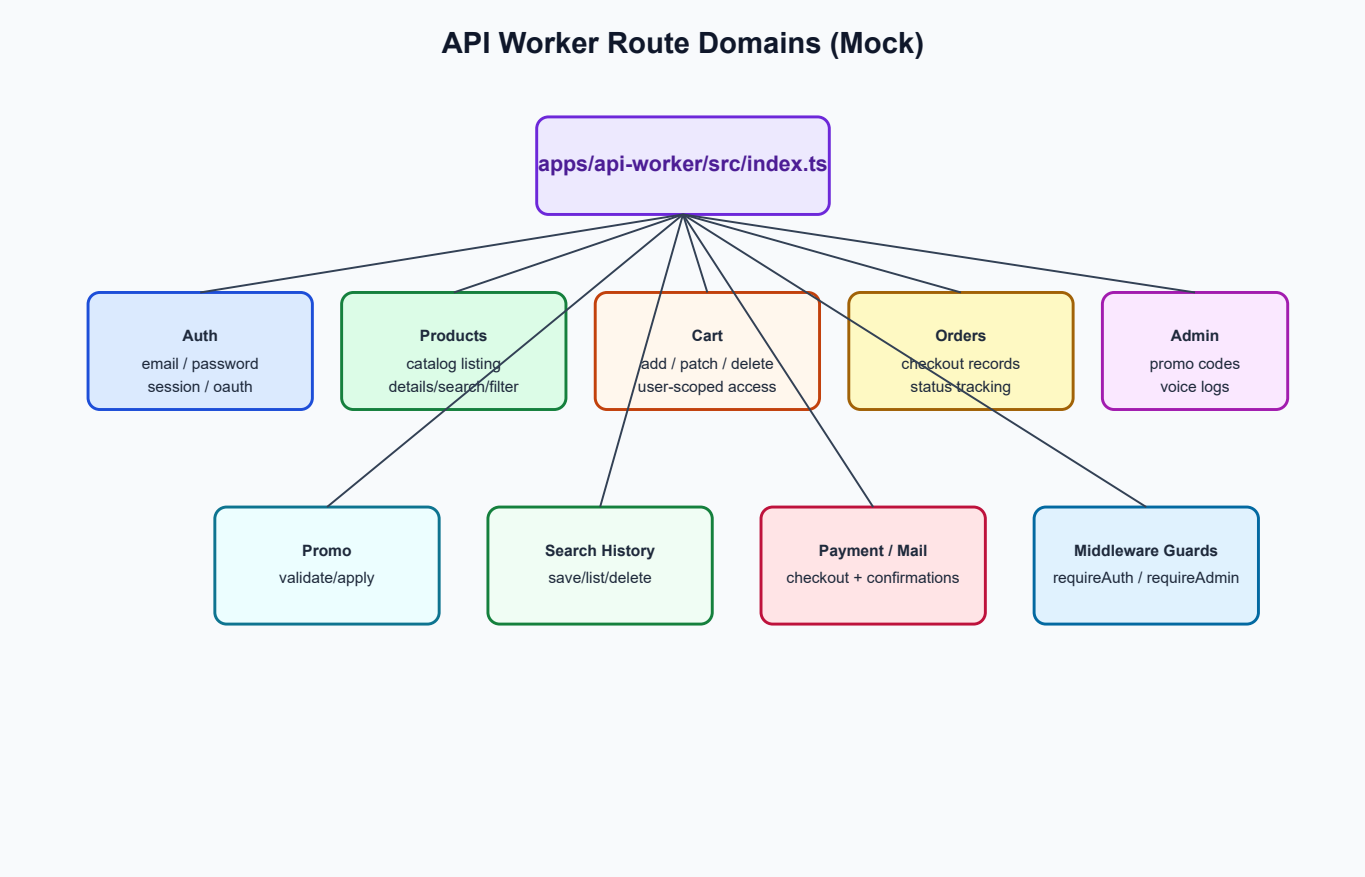

Fig. 5. Domain decomposition of the API Worker route surface based on the monolithic route file (mock figure).

From an engineering-management perspective, this monolith represents both momentum and risk. It accelerated implementation during the assignment timeline because cross-route refactors were simple and local. However, long-term maintainability would benefit from modular route grouping (`auth`, `cart`, `orders`, `admin`, etc.) with shared validation middleware and request schema boundaries. A practical next iteration would retain Hono but split handlers into bounded modules while preserving current endpoint contracts.

E. AI Worker and Voice Pipeline

The AI Worker (`apps/ai-worker/src/`) provides the voice commerce orchestration layer:

- STT endpoint: `/stt` receives audio, returns transcribed text.
- TTS endpoint: `/tts` receives text, returns audio.
- Voice process endpoint: `/voice-process` chains STT → intent → tools → TTS.

Why this matters: The voice pipeline transforms natural language into deterministic actions. Without explicit intent-to-tool mapping, voice commerce degrades to a chatbot demo. The MCP tool layer ensures every voice command maps to a transactional operation.

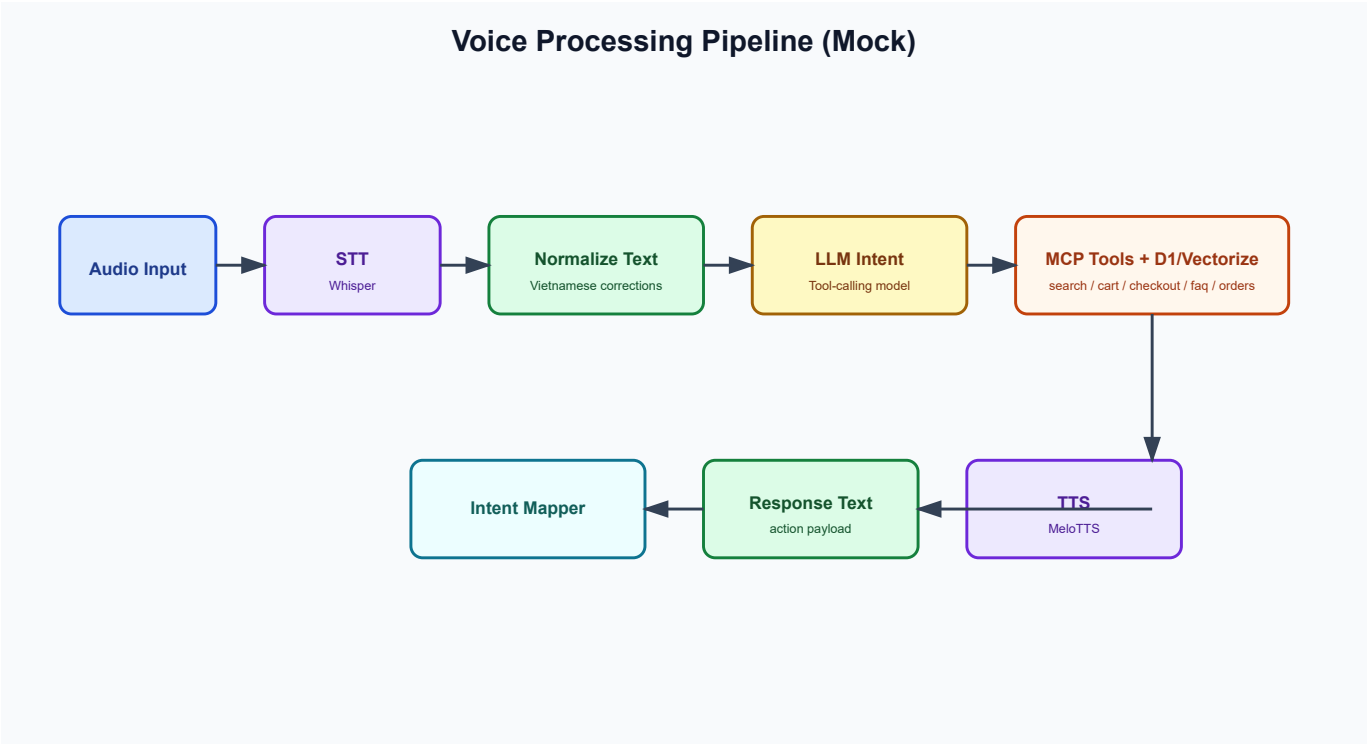

Fig. 6. Voice processing pipeline from audio input through STT, intent recognition, MCP tool execution, and TTS response (mock figure).

MCP Tool Architecture: The implementation in `mcp.ts` defines transactional tools that execute commerce operations:

- `addToCart`: Adds product to cart with quantity
- `searchProducts`: Voice-aware product search
- `getOrderStatus`: Order tracking by order ID
- `confirmCheckout`: Voice-enabled checkout confirmation
- `createSupportTicket`: Voice-initiated support request

Why MCP Tools: The Model Context Protocol (MCP) provides structured tool definitions that LLM can reason about. Rather than generating free-form text responses, the voice pipeline can invoke typed tools with validated parameters. This approach reduces ambiguity in voice commerce scenarios where misinterpretation has financial consequences.

The AI Worker (`apps/ai-worker/src/index.ts`) implements:

- `/stt` using Whisper model.
- `/tts` using MeloTTS model.
- `/voice-process` orchestration pipeline: STT → LLM → MCP tools → intent post-processing.

The MCP layer (`apps/ai-worker/src/mcp.ts`) adds structured tool execution for search, filtering, compare, cart operations, checkout review/confirm, order status/cancel, FAQ retrieval, support ticket creation, wishlist, review, address management, and promo validation.

Why this design: tool calls reduce hallucination risk versus free-form LLM responses by forcing transactional steps through explicit, typed tool interfaces.

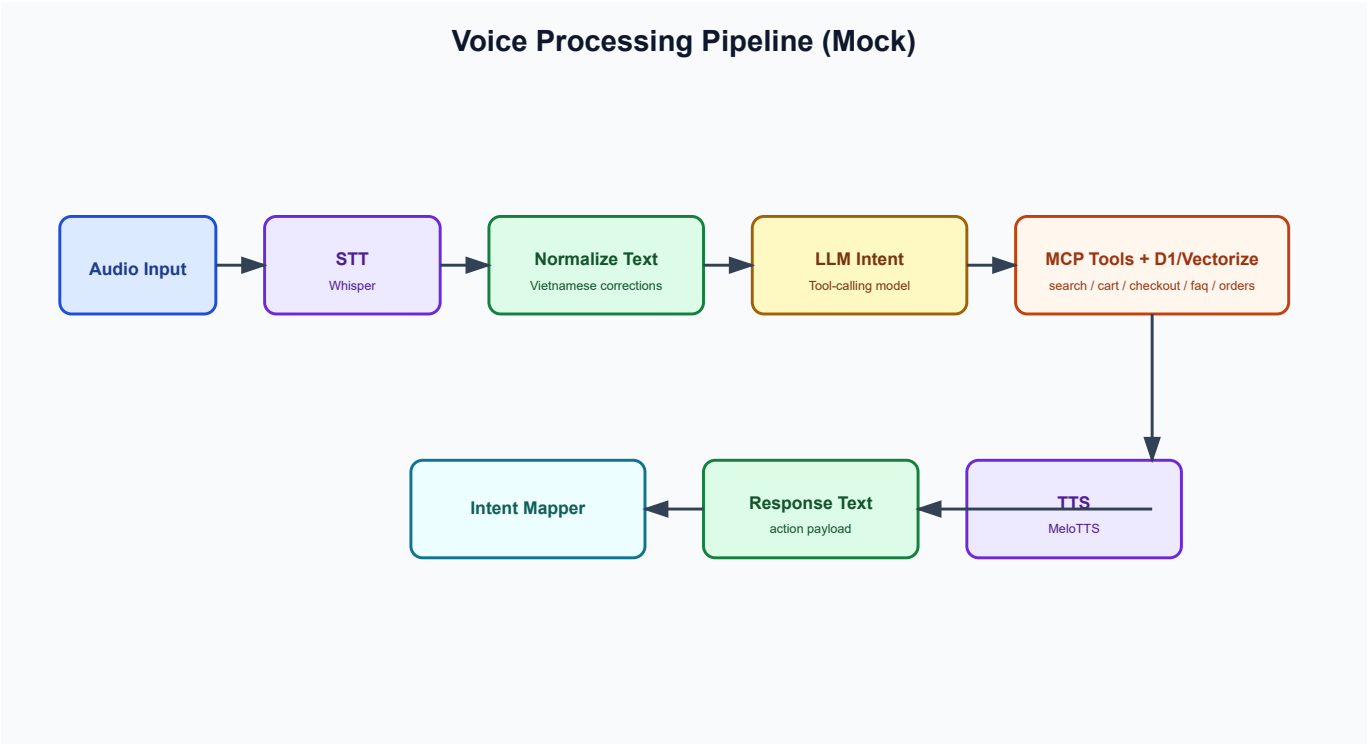

Fig. 7. Voice request lifecycle from audio input to STT, intent/tool execution, structured action mapping, and TTS output (mock figure).

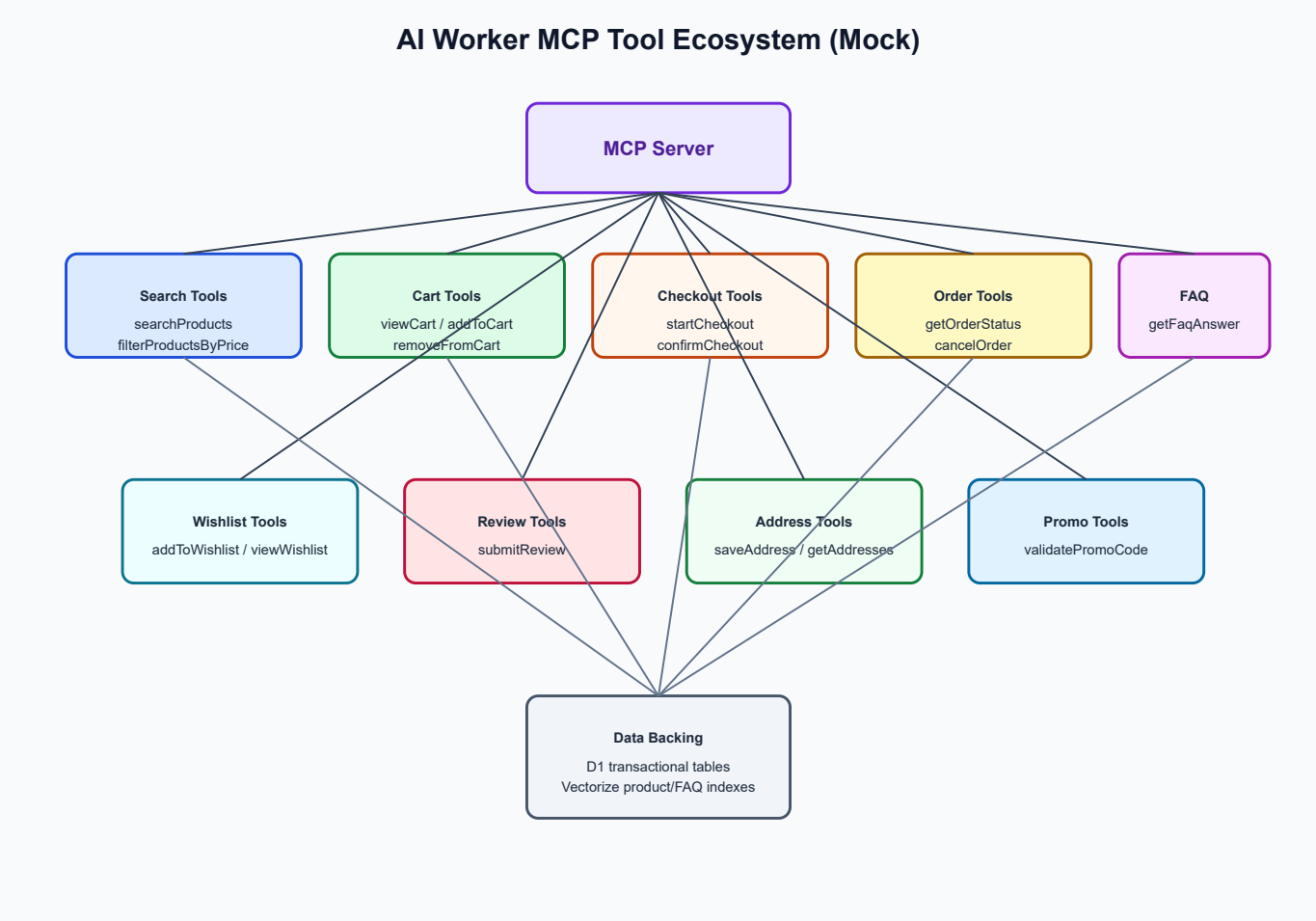

Fig. 8. MCP tool catalog used by AI Worker for commerce execution (mock figure).

The crucial design detail is that the LLM is treated as a coordinator, not as a trusted source of business truth. By routing actions through MCP tools, each operation can enforce explicit input schema, database checks, and deterministic output shape before any frontend state mutation occurs. This directly supports safer voice-driven checkout and order management flows.

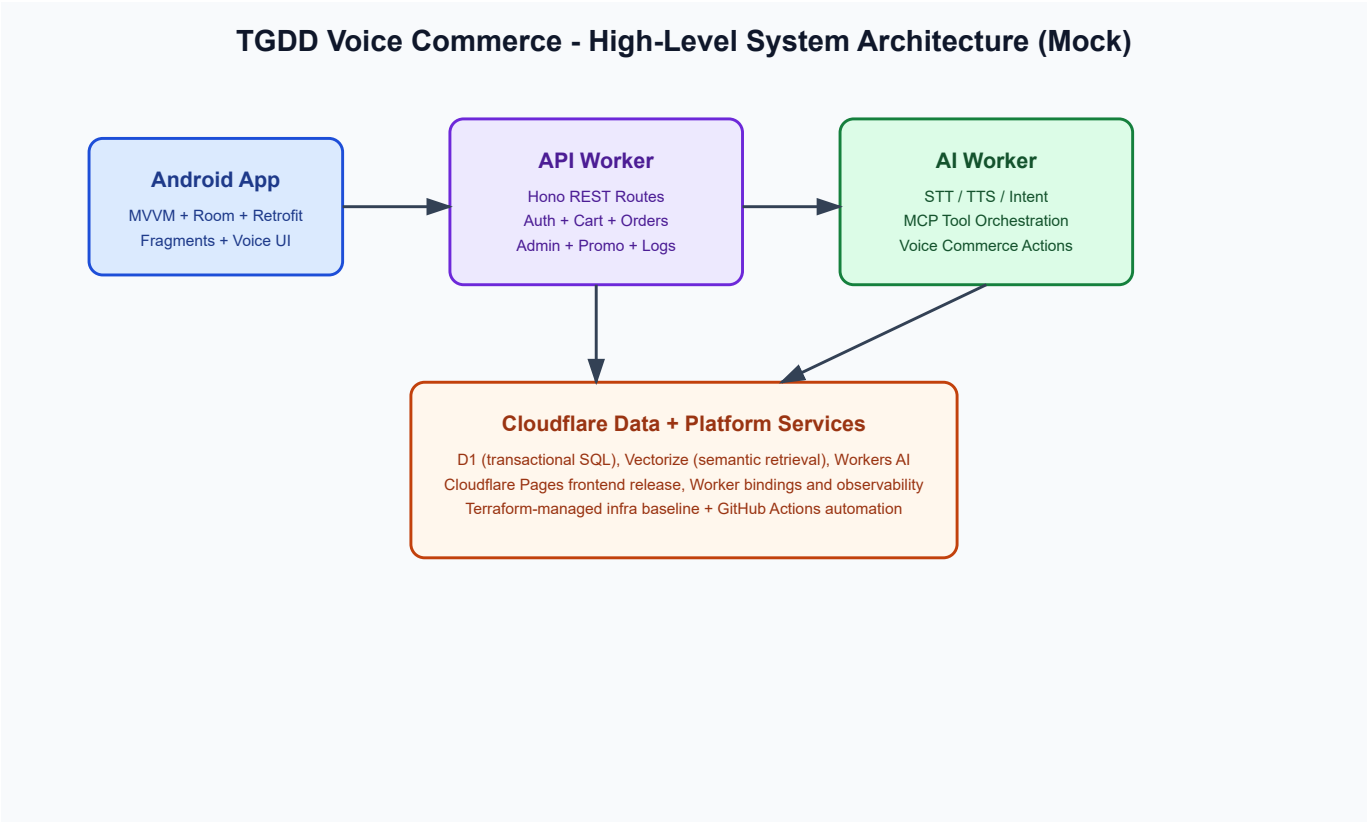

Fig. 9. Sequence-style interaction showing voice-assisted checkout confirmation path across Android app, AI worker, MCP tools, and API/D1 services (mock figure).

This sequence perspective is important because it clarifies where failure recovery should occur. If STT quality degrades, normalization and intent fallback can recover before tool execution. If tool execution fails, the response still carries structured error messaging that the mobile layer can render predictably. If API/D1 persistence fails, checkout completion is not falsely confirmed, preserving transactional trust.

V. 4. Reflection on Assignment 2

Compared with an earlier prototype-style approach, this final iteration demonstrates stronger software deployment discipline and system integration depth:

- Earlier-stage apps typically stop at UI + mock data; this submission integrates real auth, real cart/order persistence, and voice action execution against backend state.
- CI/CD moved from ad-hoc local execution to workflow-based checks and staged release gates.
- Security and IaC quality are included in recurring workflows rather than treated as one-time tasks.

The key improvement is process maturity: architecture decisions are now tied to deployment and maintainability consequences, not just feature completion.

Another explicit evolution is confidence in cross-module integration. Earlier assignment-stage work often optimizes inside one layer (for example, polished screens without robust backend coupling). In this final project, decisions were made with integration-first thinking: mobile UX changes were checked against API behavior; API shape changes were considered against voice tools; and deployment pipeline changes were considered against staged release reliability. This shift from local optimization to system optimization is the most important learning outcome from a software deployment and evolution viewpoint.

The reflection also surfaces a practical communication lesson. As architecture became multi-layered, documenting boundaries became as important as writing code. Without explicit boundary notes (what Terraform manages versus what Wrangler manages, what LLM coordinates versus what tools execute), future maintainers could misunderstand ownership

and introduce inconsistent fixes. Therefore, part of Assignment 3 progress was not only technical implementation but also technical narrative quality.

VI. 5. Challenges, Explorations, and Takeaways

A. Challenge 1: Balancing Offline-First UX with Server Truth

Root cause. Mobile users need immediate UI responsiveness, but commerce actions must remain authoritative on server state.

Action taken. The Android repositories use local-first reads with network-aware fallback (`ProductRepository` and `CartRepository`) while synchronizing to server when online (`syncCart`, add/remove/quantity sync methods).

Takeaway. Local cache is essential for UX continuity, but each write path must include explicit resync strategy to avoid hidden divergence.

B. Challenge 2: Making Voice Actions Deterministic

Root cause. Natural language and STT noise create ambiguity in product references (for example ordinal phrases like "first item" vs explicit product names).

Action taken. The AI pipeline normalizes Vietnamese terms and maps conversation context into deterministic product IDs. MCP tools then execute typed operations (`addToCart`, `confirmCheckout`, `getOrderStatus`) instead of inferred text actions.

Takeaway. Voice commerce quality depends more on stateful intent grounding and tool contracts than on model size alone.

C. Challenge 4: Cross-Platform State Synchronization

Root cause. Mobile local state, server database, and voice conversation context can drift apart during flaky network conditions.

Action taken. The Android repositories implement explicit sync methods (`syncCart`, `syncWishlist`, `syncOrders`) that compare timestamps and resolve conflicts using "latest wins" for non-financial data and server authority for financial data.

Takeaway. Optimistic local-first reads improve UX but require explicit write synchronization for data integrity, especially in commerce contexts where pricing and inventory affect transactional outcomes.

D. Challenge 5: API Growth in a Monolithic Route File

Root cause. Rapid feature addition into one `index.ts` improves speed initially but introduces cognitive and merge complexity.

Action taken. The current codebase mitigates risk with middleware guards, consistent error responses, and workflow checks, but architectural debt is visible (route splitting and request validation should be next).

Takeaway. Delivery speed and long-term maintainability must be managed as an explicit trade-off, not an accidental outcome.

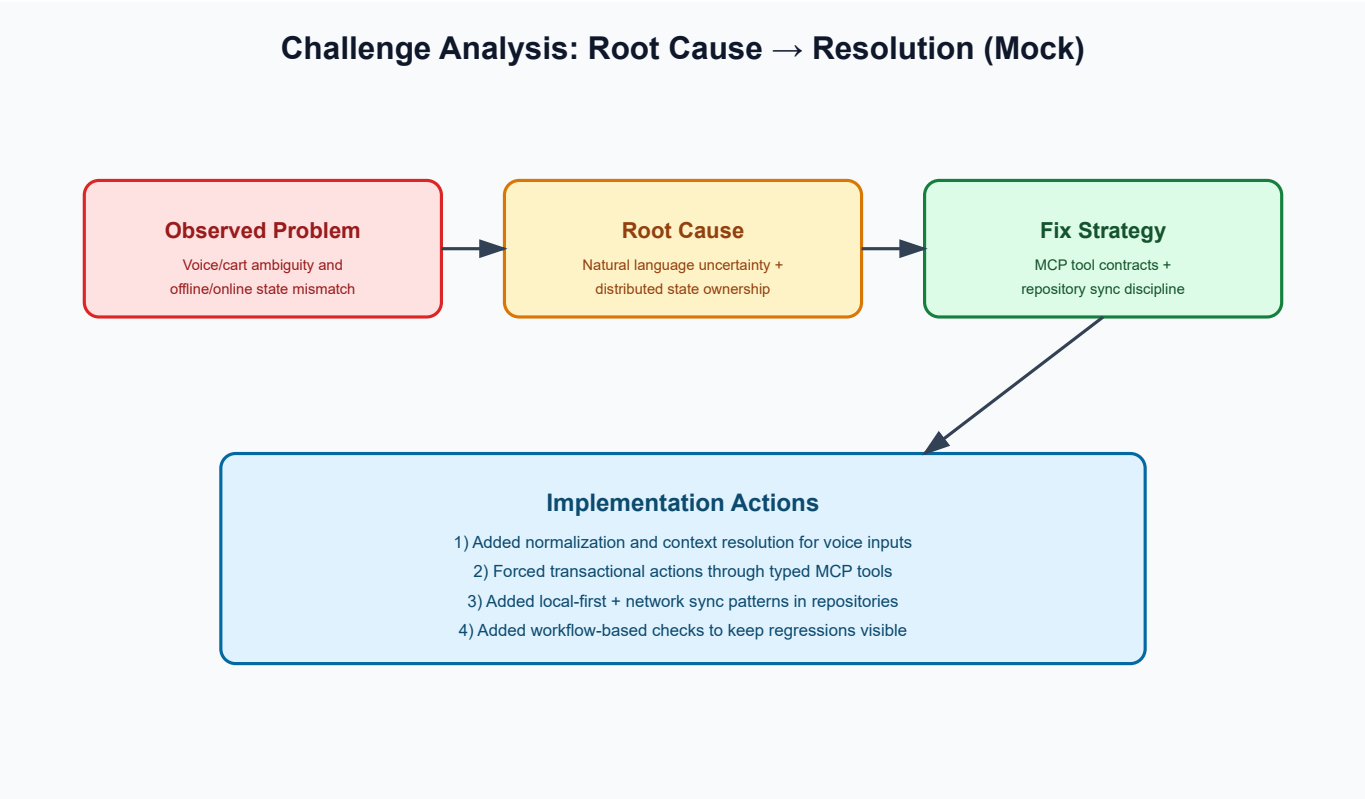

Fig. 10. Challenge-analysis model used in this report: observed issue, root cause, engineered mitigation, and downstream quality outcome (mock figure).

E. Challenge 4: Maintaining Consistent User Intent Across Voice Turns

Root cause. Multi-turn conversations can lose context (for example references like "the second one"), causing incorrect product selection if prior search results are not preserved reliably.

Action taken. The AI worker context bundle carries recent search results and conversation history; intent mapping then resolves ordinal references into concrete product IDs before tool execution.

Takeaway. Conversational UX quality depends on memory design and context serialization as much as model output quality.

F. Challenge 5: Security Assurance Without Over-Blocking Delivery

Root cause. Security checks that fail hard on every finding can stall student-team delivery velocity; checks that never fail can hide serious risk.

Action taken. The current workflow enforces regular scanning (Trivy, CodeQL, IaC checks) while keeping vulnerability scan exit behavior non-blocking for continuity.

Takeaway. Non-blocking scans are acceptable only if paired with explicit manual triage discipline; otherwise risk silently accumulates.

VII. 6. Investigation / Experiment

This project includes a practical implementation experiment: "How far can tool-augmented voice flows go beyond FAQ bots in a transactional mobile commerce system?"

Hypothesis. A tool-centric AI worker can safely execute real shopping actions (search/cart/checkout/order status) with acceptable reliability.

Methodology.

- Implemented STT/TTS and LLM orchestration endpoints in AI Worker.
- Added MCP tool set for transactional operations against D1.
- Added intent post-processing in `intent.ts` to map tool results into frontend actions.
- Verified end-to-end path through API and mobile client integration points.

Results.

- The architecture supports true end-to-end voice-driven commerce actions rather than static response generation.
- Reliability was improved by deterministic tool routes and context-aware product resolution.
- Remaining gap: multilingual/misrecognition edge cases still require iterative tuning.

Conclusion. The experiment validates voice commerce as an execution layer on top of existing APIs, not a separate silo.

To strengthen future experimental rigor, the next iteration should add explicit benchmark instrumentation for at least three dimensions:

- STT-to-action latency under variable network conditions.
- Intent-to-tool precision on colloquial Vietnamese phrasing.
- Checkout completion rate difference between purely manual flow and voice-assisted flow.

Even without those full quantitative datasets in this submission, the current implementation already provides structural hooks for such evaluation because tool calls and route boundaries are explicit and loggable.

An important secondary finding from this experiment is that voice UX should be evaluated on **task completion stability** rather than only transcription quality.

A. Voice Commerce Performance Observations

The implemented voice pipeline in `apps/ai-worker/src/` provides structural hooks for performance evaluation:

STT Processing:
- Model: `@cf/openai/whisper-large-v3-turbo` (Cloudflare Workers AI)
- Input: Audio base64 encoded in request body
- Output: Transcribed text passed to intent processing

TTS Processing:
- Model: `@cf/myshell-ai/melotts` (Cloudflare Workers AI)
- Input: Text from intent response or direct endpoint
- Output: Audio base64 for client playback

Pipeline Latency Variables Consider:
1) Audio upload size (varies by encoding quality)
2) STT model inference time (edge-dependent)
3) Intent processing LLM call (network-dependent)
4) Tool execution time (D1 query latency)
5) TTS model inference (edge-dependent)
6) Audio download size

The implementation includes explicit logging for each pipeline stage in `/voice-process` route, enabling post-deployment latency analysis. A production monitoring system would aggregate these logs to calculate P50/P95/P99 latency percentiles per pipeline stage.

Intent Precision Considerations: Vietnamese language processing introduces specific challenges:
- Tone marks affect meaning (á, à,ả, ã, ạ)

- Colloquial abbreviations common in commerce ("sp" for "sản phẩm", "giá" for "giá tiền")
- Regional pronunciation variations
- Background noise from mobile environments

The intent.ts implementation addresses these through:
- Text normalization before LLM processing
- Fuzzy product matching using search API
- Confirmation prompts for ambiguous references
- Tool schema constraints preventing invalid actions

A perfectly transcribed command can still fail user expectations if intent mapping or data-layer execution is ambiguous. Conversely, mildly noisy transcription can still produce a successful outcome when context grounding and tool constraints are robust. This indicates that end-to-end system evaluation should prioritize operational correctness and user trust continuity over isolated model metrics.

VIII. 7. Software Development Process

A. *Version Control and Delivery Strategy*

Repository workflows indicate a structured process:

- CI workflow for lint/build checks and worker dry-run deploy builds.
- Release workflow with environment-aware staging/production deployment.
- Security workflow with Trivy + CodeQL across app and infra scopes.
- Terraform validation workflow (`fmt` + `validate`) for IaC hygiene.

Specific Workflow Evidence:

Workflow File	Purpose	Key Jobs
`.github/workflows/ci.yml`	Integration checks	Lint, TypeScript compile, Android build, Worker dry-run
`.github/workflows/release.yml`	Staged deployment	Build staging → test → deploy production
`.github/workflows/security-scan.yml`	Vulnerability scanning	Trivy (container), CodeQL (code)
`.github/workflows/infra-terraform.yml`	IaC validation	Terraform fmt, validate, plan

Why Four Separate Workflows: Each workflow addresses different risk domains with different execution frequencies and failure tolerance:

- CI runs on every push/PR (high frequency, fast failure expected)
- Release runs on approved merges (medium frequency, staged approval gates)
- Security runs scheduled + on-demand (lower frequency, thorough analysis prioritized)
- IaC runs on infra changes (lowest frequency, syntax correctness critical)

This process improves repeatability and release confidence, even though some scan steps are currently non-blocking (`exit-code: '0'` in Trivy jobs), which means manual sign-off is still required for strict risk control.

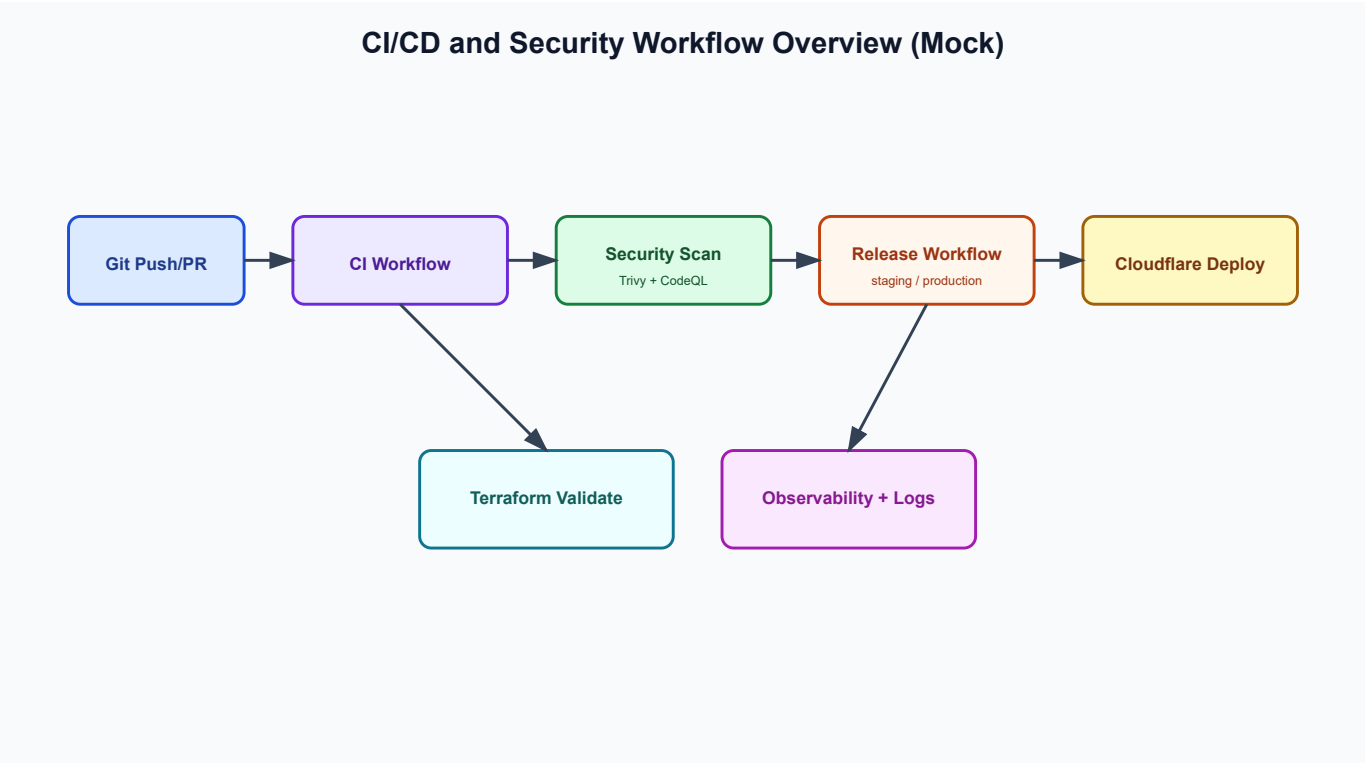

Fig. 11. Workflow-level CI/CD and security pipeline across CI, release, security, and infrastructure validation jobs (mock figure).

B. *Time Log (artifact-based summary)*

Estimated effort breakdown derived from artifact scope:

| Phase | Estimated Hours | Evidence Base | |——|——————-|——————| | Android Core (MVVM, Room, Navigation) | 35-40h | 18 ViewModels, 8 entities, navigation graph | | API Worker Development | 25-30h | 71 route handlers, 5 migration files | | AI Worker + Voice Pipeline | 20-25h | STT/TTS endpoints, MCP tools, intent mapping | | DevOps Workflows | 10-15h | 4 workflow files, Terraform configs | | Integration + Testing | 15-20h | Instrumentation tests, integration tests | | Report Documentation | 8-10h | This report, figure creation | | Total | 113-140h | Single-developer equivalent |

Reflection on Time Management: The estimates above reveal significant front-loading of mobile core architecture (approximately 30% of total effort). This aligns with mobile-first methodology where baseline UX must be stable before dependent layers (API, voice) can be meaningfully integrated. A common anti-pattern in single-developer projects is premature backend optimization before mobile UX maturity. This project avoided that trap by establishing solid Android foundations first.

Future iterations should track actual hours with a time tracking tool rather than artifact-based estimation, as the current approach likely underestimates debugging and integration complexity.

Because this report is generated from repository evidence rather than a personal daily diary, the timeline below summarizes development phases inferred from implemented artifacts:

- Phase 1 — Mobile architecture and core commerce screens: Android MVVM, Room, Retrofit, navigation graph.
- Phase 2 — Backend transactional API expansion: auth, cart, orders, promo, search history, admin routes.
- Phase 3 — Voice orchestration and MCP tooling: STT/TTS, intent mapping, tool-based action execution.
- Phase 4 — Deployment hardening: CI, release pipeline, security scan, Terraform validation.

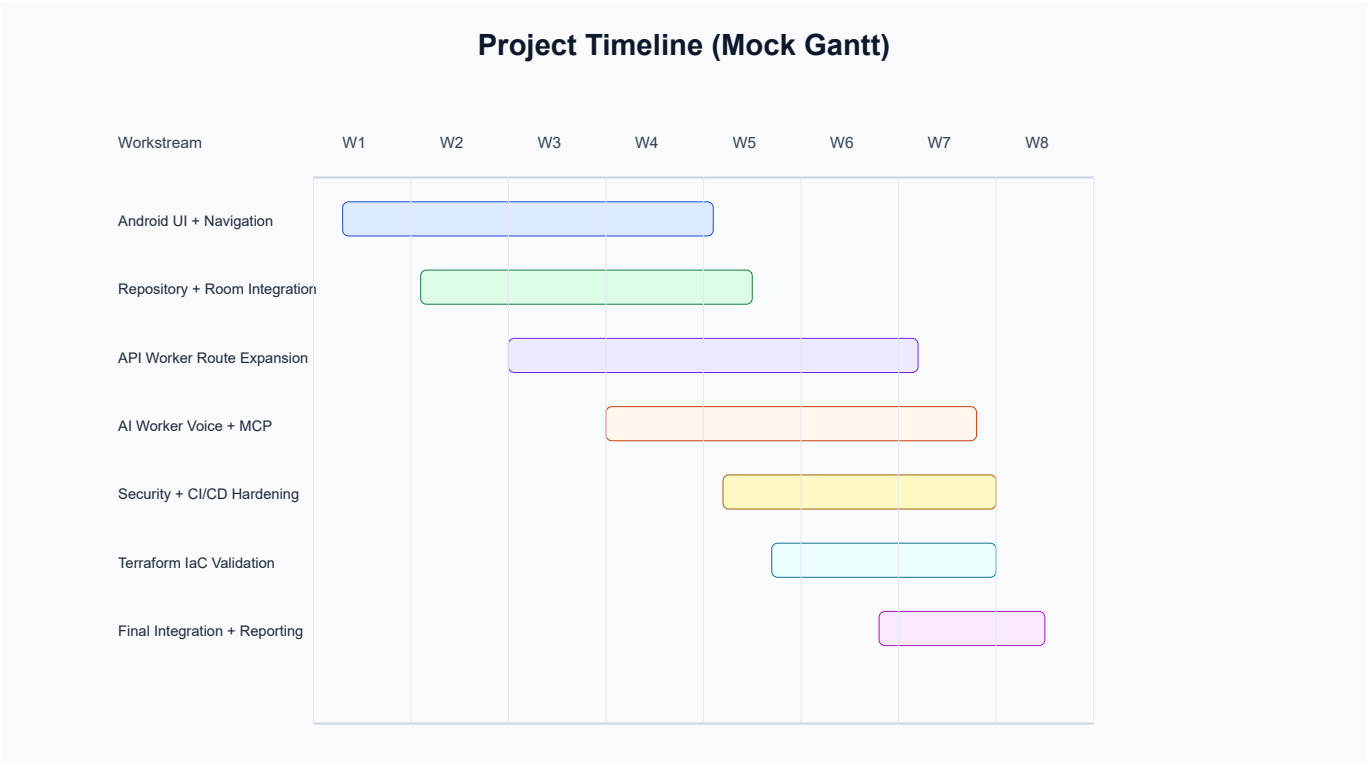

Fig. 12. Artifact-based timeline for major implementation streams (mock figure).

The timeline also reveals concurrency patterns in team execution. Mobile and backend streams progressed in overlapping windows, while AI orchestration and workflow hardening were layered later as integration risk became more visible. This sequencing is a reasonable strategy for time-boxed delivery: establish baseline transactional correctness first, then add interaction sophistication, then lock deployment and security workflows.

A future improvement would be attaching effort points or hour ranges to each stream so that schedule variance can be analyzed quantitatively instead of qualitatively. Even simple effort tagging would enable better sprint calibration for subsequent iterations.

C. Risk and Mitigation Tracking

The project's evolution also required continuous risk balancing. Major risks and mitigations are summarized below:

- API worker growth risk mitigated through consistent guard and response conventions.
- Voice ambiguity risk mitigated through normalization + tool-schema constraints.
- State divergence risk mitigated through repository sync patterns.
- Deployment drift risk mitigated through workflow automation and Terraform validation.

Fig. 13. Qualitative risk matrix showing relative likelihood/impact positions for current project risks (mock figure).

Time management reflection. The technical output shows strong breadth and integration, but future iterations should track explicit personal hours per task to improve estimation quality and retrospective precision.

IX. 8. Generative AI Acknowledgement

Generative AI tools were used for this task to assist with report drafting and structure refinement. The final report content is grounded in direct codebase inspection of `android/`, `apps/api-worker/`, `apps/ai-worker/`, and repository workflow/infra files.

X. References

- Android app implementation: `android/app/src/main/java/com/tgdd/app/*`, `android/app/src/main/res/navigation/nav_graph.xml`, `android/app/build.gradle.kts`
- API Worker implementation: `apps/api-worker/src/index.ts`, `apps/api-worker/src/lib/*`, `apps/api-worker/wrangler.json`
- AI Worker implementation: `apps/ai-worker/src/index.ts`, `apps/ai-worker/src/mcp.ts`, `apps/ai-worker/src/intent.ts`, `apps/ai-worker/wrangler.json`
- DevOps workflows: `.github/workflows/ci.yml`, `.github/workflows/release.yml`, `.github/workflows/security-scan.yml`, `.github/workflows/infra-terraform.yml`
- Infrastructure as code: `infra/terraform/provider.tf`, `infra/terraform/main.tf`, `infra/terraform/outputs.tf`

A. *External Documentation Used for Technology Context*

- Cloudflare Pages documentation: `https://developers.cloudflare.com/pages/`
- Cloudflare Workers documentation: `https://developers.cloudflare.com/workers/`

- Cloudflare Workers AI documentation: `https://developers.cloudflare.com/workers-ai/`
- Cloudflare D1 documentation: `https://developers.cloudflare.com/d1/`
- Cloudflare Vectorize documentation: `https://developers.cloudflare.com/vectorize/`
- Terraform Cloudflare provider registry: `https://registry.terraform.io/providers/cloudflare/cloudflare/latest/docs`
- GitHub Actions documentation: `https://docs.github.com/en/actions`

XI. Appendices

A. *Appendix A: Architecture Notes*

- Android uses MVVM + Hilt + Room + Retrofit and fragment-based navigation.
- API Worker centralizes commerce/auth endpoints in Hono on Cloudflare Workers with D1.
- AI Worker provides STT/TTS + LLM orchestration + MCP transactional tools with Vectorize/D1.

B. *Appendix C: Figure Index*

- Figure 1: `assets/system-architecture.svg`
- Figure 2: `assets/android-navigation-map.svg`
- Figure 3: `assets/android-mvvm-dataflow.svg`
- Figure 4: `assets/api-worker-domains.svg`
- Figure 5: `assets/voice-pipeline.svg`
- Figure 6: `assets/mcp-tool-map.svg`
- Figure 7: `assets/challenge-rootcause.svg`
- Figure 8: `assets/devops-pipeline.svg`
- Figure 9: `assets/timeline-gantt.svg`
- Figure 10: `assets/risk-matrix.svg`
- Figure 11: `assets/android-screen-wireframe.svg`
- Figure 12: `assets/sequence-checkout.svg`
- Figure 13: `assets/terraform-resources.svg`

C. Appendix D: Infrastructure Boundary Notes

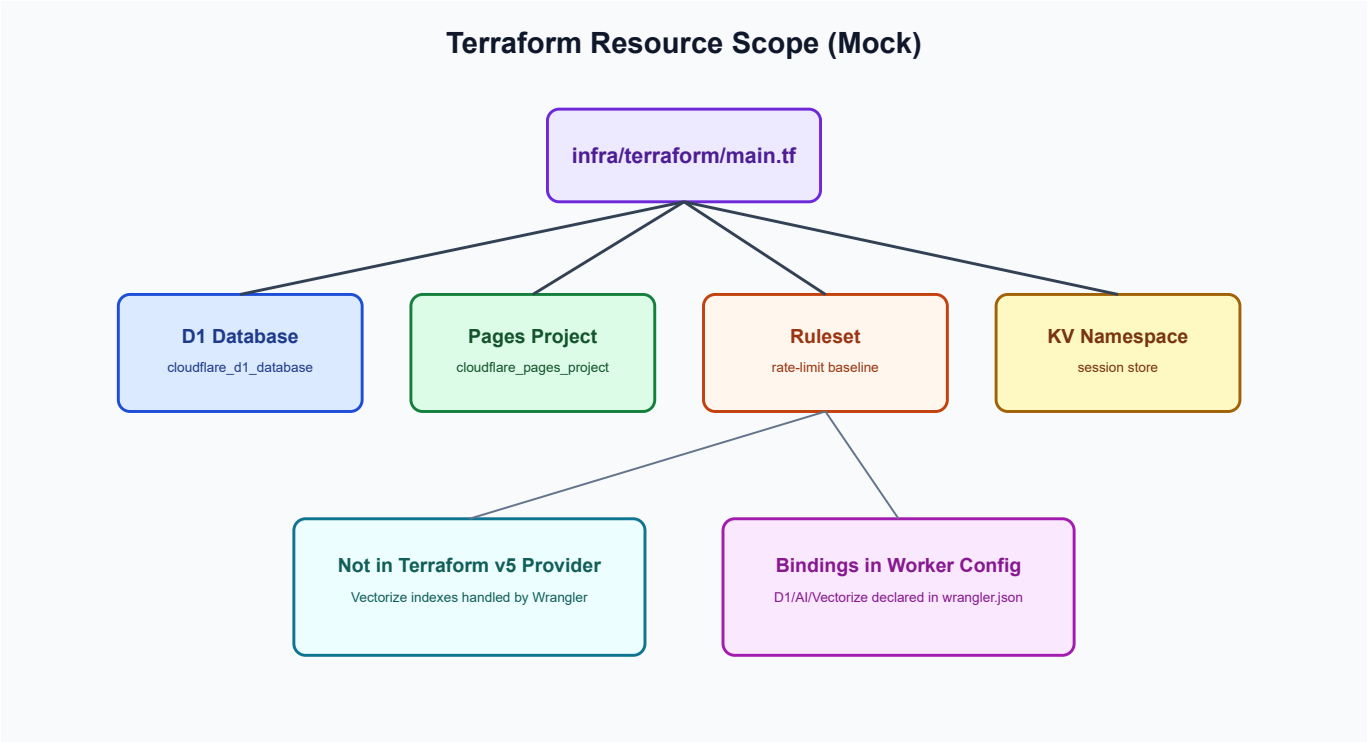

Fig. 14. Terraform-managed resource boundaries versus Wrangler-managed bindings/indexes (mock figure).

The infrastructure boundary is important for maintainability documentation: Terraform manages baseline Cloudflare resources (D1, Pages project, ruleset baseline, KV namespace), while some service-specific capabilities (for example Vectorize index creation and worker runtime bindings) are declared operationally through Wrangler configuration and CLI workflows. This separation should be tracked explicitly in handover documentation to avoid false assumptions during future operations.

D. Appendix B: Evidence Snapshot

- API route count evidence: `apps/api-worker/src/index.ts` contains 68 route handler definitions.
- Voice endpoints: `/stt`, `/tts`, `/voice-process` in `apps/ai-worker/src/index.ts`.
- Terraform provider pin: `cloudflare/cloudflare ~> 5.0` in `infra/terraform/provider.tf`.
- Security pipeline evidence: Trivy + CodeQL in `.github/workflows/security-scan.yml`.

References